

Lab 10: Mouse Input Handling, Multimedia Processing, Network Programming, and Database Connections

Setting Up

1. Download the zip file for lab-10 on Brightspace.
2. Unzip the file to a nice working directory where you usually do your work
3. Enter the unzipped directory. Consider loading it in Visual Studio Code, Eclipse, or the terminal
4. Review the lecture outline files: `mouse-input-handling.pdf`, `multimedia-processing.pdf`, and `database-connections.pdf` to make sure you understand the concepts covered in the examples.

Exploring the Code Examples

5. **Plotting Circles with the Mouse:** Compile and run the program `circlePlotter.java`. Notice what `.class` files were produced. Click inside the window to see circle appear at the location you click. Open the source file and explore the code.
 - a. Which class contains the main class?
 - b. On which class are the circles drawn?
 - c. What is the type of the class on which the circles are drawn
 - d. What Interface does the Panel implement that allows it to process mouse input?
 - e. What function in the code draws the circle?
 - f. What line determines the size of the circle?
 - g. What variable contains all the information about the circles to be draw?
 - h. What is the name of the function that actually draws the circles?
 - i. What methods needed to be implemented to satisfy the Interface?
 - j. Which overridden interface method responds to the mouse clicks that cause the circle to be drawn?
6. **Moving Squares with the Mouse:** Compile and run `SquarePlotter.java`. Notice what `.class` files were produced. Click inside the program to see squares painted at the location of the mouse. Click and drag one of the squares. Notice that the square moves along with the mouse until you release the left mouse button. Explore the source code.
 - a. Notice the similarities between the `circlePlotter` program and the `SquarePlotter` program. What differences do you see?
 - b. What Interfaces are implemented for this program?
 - c. What are the names of all of the Interface methods that were overridden with new code that will affect the behavior of the program.
 - d. What is the name of the overridden method that enables you to move squares around the screen?
 - e. What is the purpose of overriding the `mouseReleased` method?
 - f. What kind of behavior does the flag `insideExisting` determine in the program? What happens when the value is false? What happens when the value is true? What behavior must the user be doing or not doing for the value to be true?
 - g. Which methods belong to the `MouseMotionListener` interface?

7. Rotating and Resizing With the Mouse: Compile and run the program

`SquarePlotterAdvanced.java`. Make special note of the additional behaviors the program allows.

Practice manipulating the squares in this program. Inspect the code.

- What two new manipulations do you have available in this program from the previous square program?
- What lines of code make these new manipulations possible? Add print statements to make sure you are correct and watch messages print to stdout as you resize and rotate a square.
- Have any new interfaces been implemented and methods overridden?

8. Introducing JavaFX: Compile and run the program `HelloJavaFX` by following the instructions in the `READ.md` file. Make sure you include the `-module-path` flag and the location of the `javafx-sdk-24/lib` provided with your lab download. You will also need the `-add-modules` flag and the `javafx.controls` module listed along with the flag. After the program is running, inspect the code.

- What prints to the screen for this program?
- What happens to the text as you resize the window?
- What packages are imported into this program and what do they do?
- What is the superclass of this program?
- What method of the superclass is overridden to make execution happen?
- What line sets the text that appears in the window?
- What line sets the text that appears in the menu bar of the program?
- What happens if you comment out line 16?

9. Playing Videos With JavaFX: Compile and run the program `VideoPlayer.java` by following the instructions in `README.md`. Make sure you have the appropriate flags mentioned in part 8, but also include the appropriate modules specific to this program listed in the readme file. Feel free to load the video `test.mp4` provided with this lab to test the program or any other mp4 file that you have on hand. After you successfully play a video using the program, exit the program and inspect the code.

- Is this program using the JavaFX library?
 - What packages are imported and used by this program?
- c. Which packages are not from JavaFX?
- What object provides a UI for the user to pick a video?
 - What object actually plays the video?
 - In terms of program execution, what is the relationship between the methods: `start()`, `launch()`, and `main()`?

Feel free to watch the provided video `test.mp4` in a regular video player to learn about databases if you are curious about how they work and haven't yet taken a databases class.

10. Higher Dimensional Data Representation: Compile and run the programs `Scatter3DPlot.java`, `Scatter3DPlotFromFile.java`, `Scatter3DPlotWithTime.java`, and `Scatter3DPlotWithTimeExport.java` in that order. Feel free to use the provided `sample_data.csv` and `sample_data_4d.csv` file as input for the programs as needed, or feel free to generate new data from the programs `Generate3DData.java` and `Generate4DData.java`. Instructions for running these programs, which use JavaFX can be found in the `README.md` file. When you export your time data to a video file, you can use any player to watch the video, including the program `VideoPlayer.java` that you were using earlier, though you might find the video zoomed in.

- a. Use the mouse wheel to zoom in and out in `Scatter3DPlot.java` and click and drag to revolve around the origin. Find the code that makes this possible in the program. What two methods in the code make this possible and what lines are they located at?
- b. Look at the generated 3d data in `sample_data.csv`. How many columns are there for each row of data? What does each column represent? What does each row represent? How many spheres are present in the program when loading from this file?
- c. See if you can find another file online with the same format, or have another program generate one, or even create your own manually. Find something that is more than just random dots that follows some kind of pattern or shape. Load it into the program and explore.
- d. After loading the file `sample_data_4d.csv` into `Scatter3DPlotWithTime.java` and pressing play, analyze the 4d data. How many columns are in this data? What does each represent? Which column determines when a sphere appears? What code did you look at to determine this?
- e. In the same program, which variable holds all of the points? What is its type?
- f. Run the program `Scatter3DPlotWithTimeExport.java` load the data from `sample_data_4d.csv`. Press the button to capture video. The dots should begin populating based on their time value. The recording of the video will not be completely produced until the full time has passed. While it is recording, use your mouse to move around zooming in and rotating to capture some cool angles in the video. When the video is finally produced, play it on a video player and watch as your motion is replayed. Study the code to see if you can make sense of how the video was captured.

11. Database Programming: Compile and run the programs `PostgresConnectionExample.java` and `PostgresQueryExample.java`. When executing the programs, make sure you remember to include the `-cp` flag to set the classpath so that the JVM can find your postgres driver provided with this lab. An example of the command is included in the `README.md` file. When you run the program it is okay if you don't take time to run a database daemon on your machine and your execution of the program hits the error path. Though if you have the time to set up a database, it could be a bit more fulfilling.

- a. What packages are included in the database programs?
- b. Notice the information needed to connect to the database. What is the protocol, hostname, port, schema, username, and password for the database being connected to in the program.
- c. What command is run in each program to retrieve information from the database?
- d. What is a `ResultSet` and what is its role in retrieving information from the database?
- e. What lines of code execute if the database connection fails?

12. Network Programming: Compile the programs `chatClient.java` and `chatServer.java`. Open 4 command line windows. Run the `chatServer` program in one window. Run `chatClient` in the other 3 windows. Type a message in any given `chatClient`, press ENTER, and then view all of the other windows. Notice what has happened. Repeat this process many times until you are satisfied that you see how the program behaves. Inspect the code of each program.

- a. What printed to the server window versus the chat windows whenever a message was created by one of the clients?
- b. Did all of the clients receive a message? Is there a way to tell which client sent the message?
- c. What packages are imported for these programs?
- d. What port are the applications using to communicate?
- e. What is the type of the object that facilitates the communication?
- f. What method in the server program is responsible for transmitting messages to all the clients?
- g. What method is responsible on the clients for listening to and receiving messages from the server?

h. Alter the pair of programs so that each client gets a unique name or id. Have the server indicate which client sent a message. Save these altered programs to your lab-09 directory in your course repository.

13. **Games:** Using what you've learned from the previous examples in this lab, run the programs `HexagonGame.java` and `FirstPersonDemoWithEnemies.java`. Study in great detail how they work. Make some modifications to them to verify what you think you know. Have fun changing the programs as you wish.

14. **Networked Games:** In similar fashion to how you ran the pair of programs `chatClient.java` and `chatServer.java`. See if you can run `HexagonGameClient.java` and `HexagonGameServer.java` and get a 2-player networked game going. When you are finished with that pair of programs, try to do the same with two instances of `HexagonGameP2P.java`. Unlike the client-server version, you will use the same program to get the multiplayer game going in the P2P version. Play around until you are satisfied with the programs. Also study the game `DeathmatchGame.java` which similar to the P2P version of Hexagon, runs both players on the same program with neither running instance of the program being the server that controls the game messaging. It will be helpful if you run the instances of the game side by side to see what each player sees. For maximum fun, you might want to try and run the game on two computers connected to the same LAN to see if you can get it working. Study the networking code and be sure you understand the logic of how the game works.

a. Take which ever game or set of programs you want (Hexagon, Deathmatch, or Bomberman) and extend it so that the game can have an arbitrary number of players playing against each other on a LAN. The game should be able to work with more than just two players. Save the new program in your lab-09 repository.